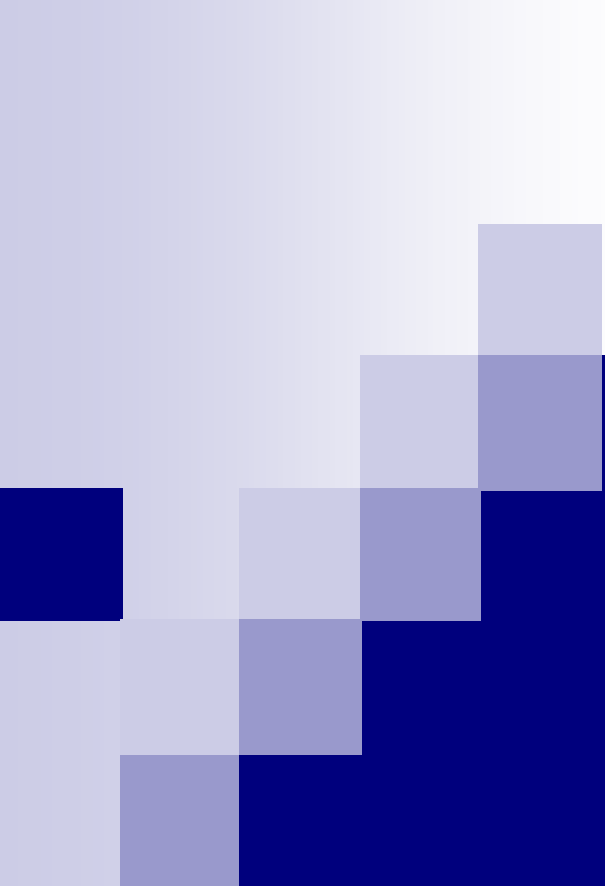




Software Inspections - I



Lecture # 15

Inspections - 1

- An inspection is a rigorous team review of a work product by peers of the producer of the work product
- The size of the team will vary with the characteristics of the work product being inspected; e.g., size, type

Inspections - 2

- The primary purpose is to find defects, recording as a basis for analysis on the current project and for historical reference and for improvement for future projects, analyzing them, and initiating rework to correct the defects
- Direct fault detection and removal

Inspections - 3

- Inspections are most effective when performed immediately after the work product is complete, but they can be held any time the work product is deemed ready for inspection

Inspections - 4

- Inspections are critical reading and analysis of software code or other software artifacts, such as designs, product specifications, test plans, etc
- Inspections are typically conducted by multiple human inspectors, through some coordination process. Multiple inspection phases or sessions may be used

Inspections - 5

- Faults are detected directly in inspection by human inspectors, either during their individual inspections or various types of group sessions
- Identified faults need to be removed as a result of the inspection process, and their removal also needs to be verified

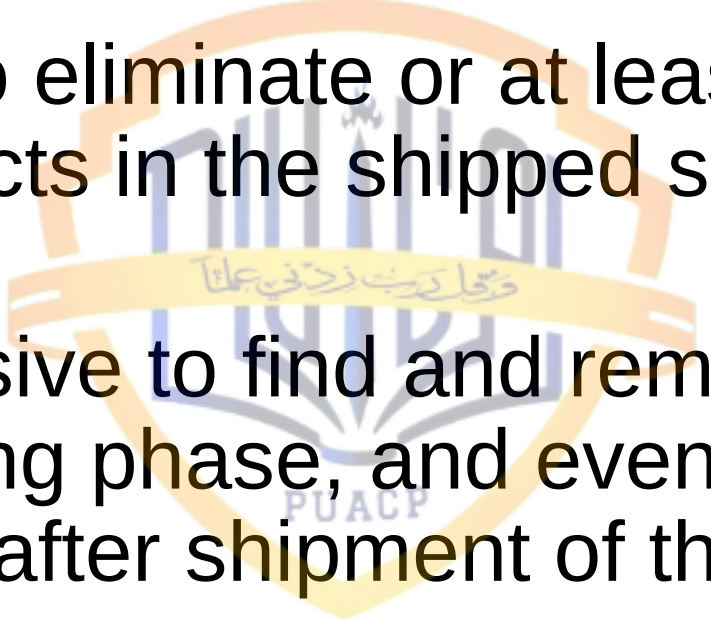
Inspections - 6

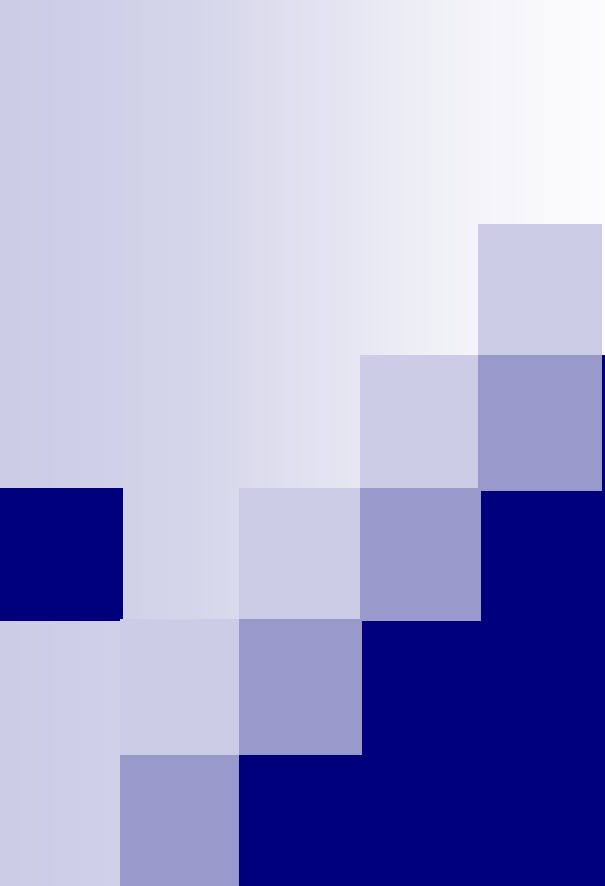
- The inspection processes vary, but typically include some planning and follow-up activities in addition to the core inspection activity
- Developed by Michael Fagan at IBM and were first reported in public domain in 1976

Inspections - 7

- Inspections remove software defects at reduced cost
- Inspections enable us to remove defects early in the software life cycle, and it always cheaper to remove defects earlier in than later in the software life cycle

- 
- We know that defects are injected in every software life cycle activity
 - We remove some of these defects in testing activities after code is completed
 - We also know that all defects are not removed at shipment time, and these are known as latent defects

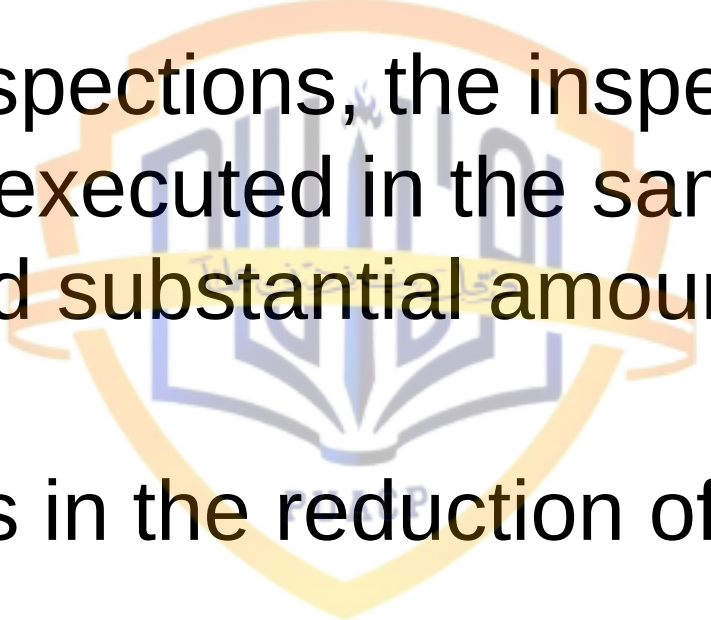
- 
- 
- We want to eliminate or at least minimize latent defects in the shipped software product
 - It is expensive to find and remove defects in the testing phase, and even more expensive after shipment of the software
 - We can use inspections to reduce these costs and improve the timelines also

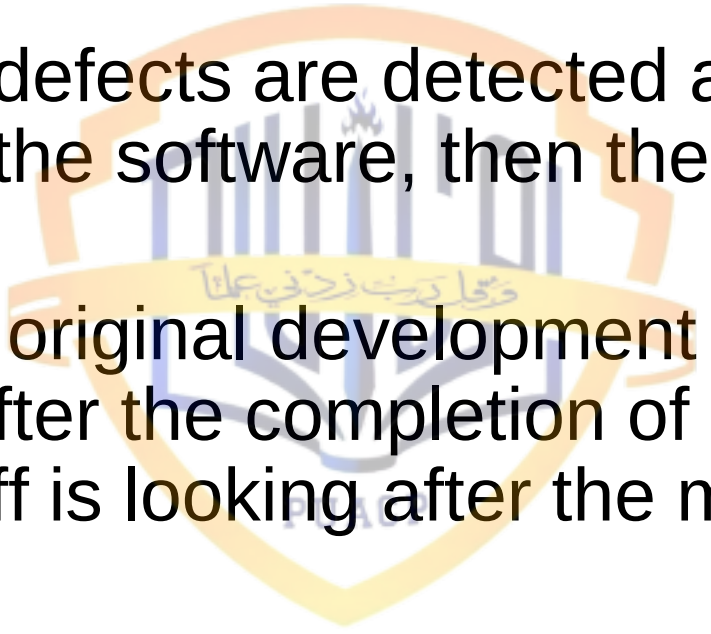


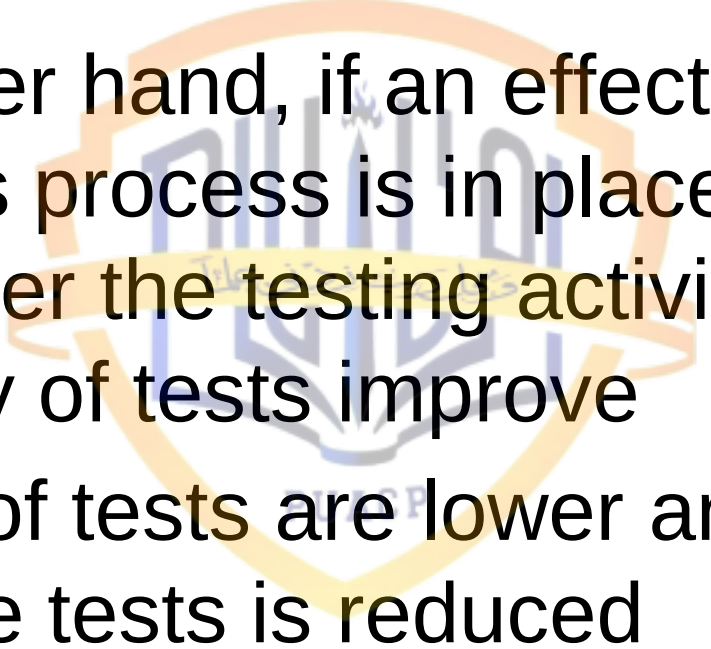
How Defect Removal is Cheaper for Inspections as Compared to Software Testing



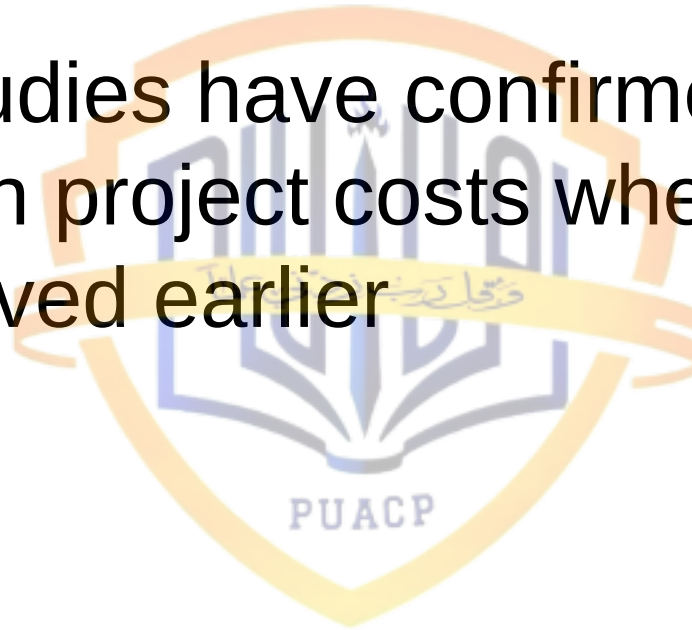
- 
- During testing, defects are found, then the programmers are notified of their presence, who will recreate the defects under the similar circumstances, fix them, re-test the software and re-integrate the software module, which were affected

- 
- 
- While in inspections, the inspection process is executed in the same life cycle activity, and substantial amount of rework is avoided
 - This results in the reduction of costs

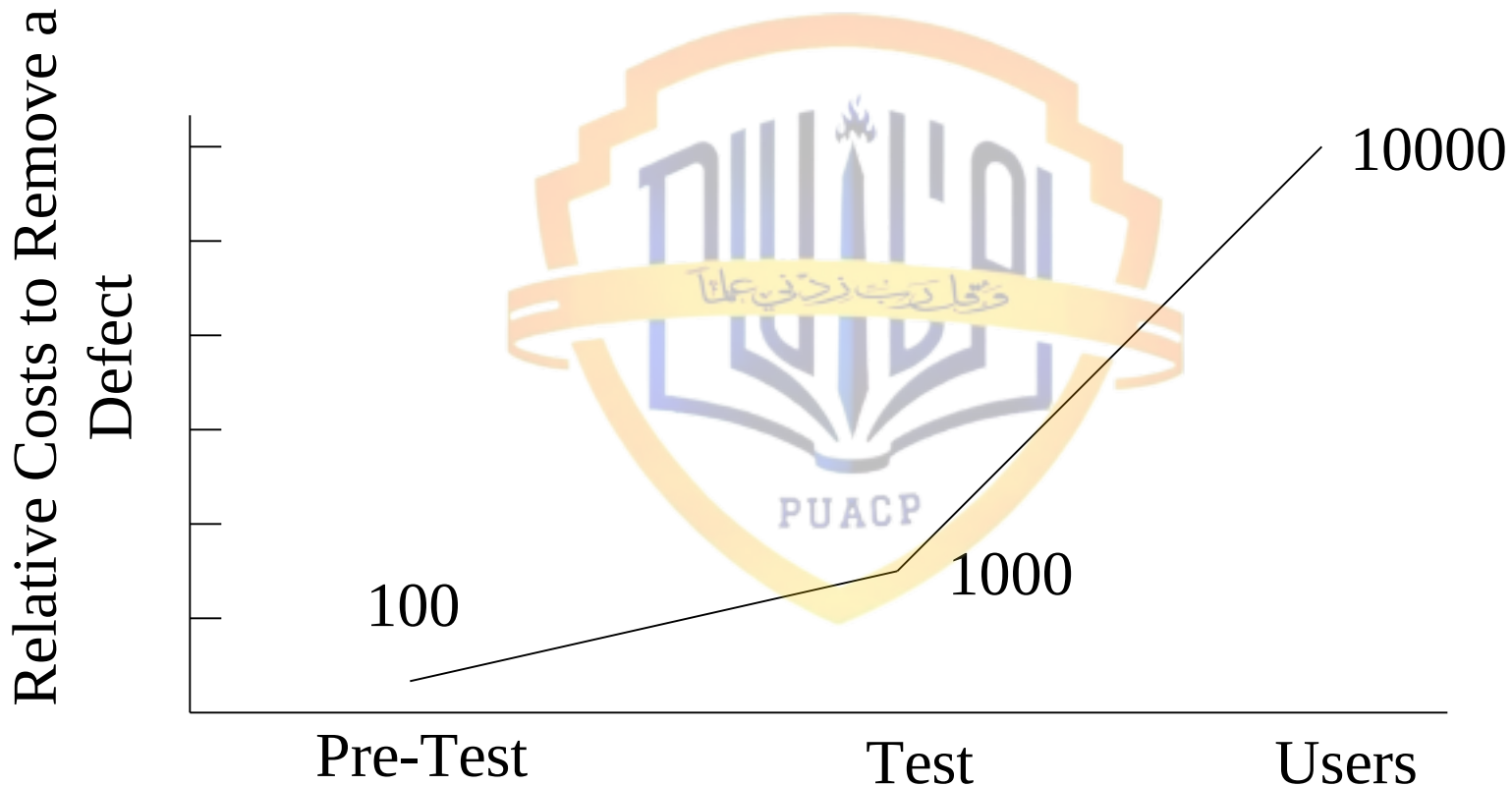
- 
- 
- If and when defects are detected after the shipment of the software, then these costs are even higher
 - Many times, original development team is disbanded after the completion of the project and new staff is looking after the maintenance activity
 - These people are usually not fully aware about the project
 - This can result in unplanned expenses for the software development company

- 
- 
- On the other hand, if an effective software inspections process is in place, fewer defects enter the testing activity and the productivity of tests improve
 - The costs of tests are lower and the time to complete tests is reduced

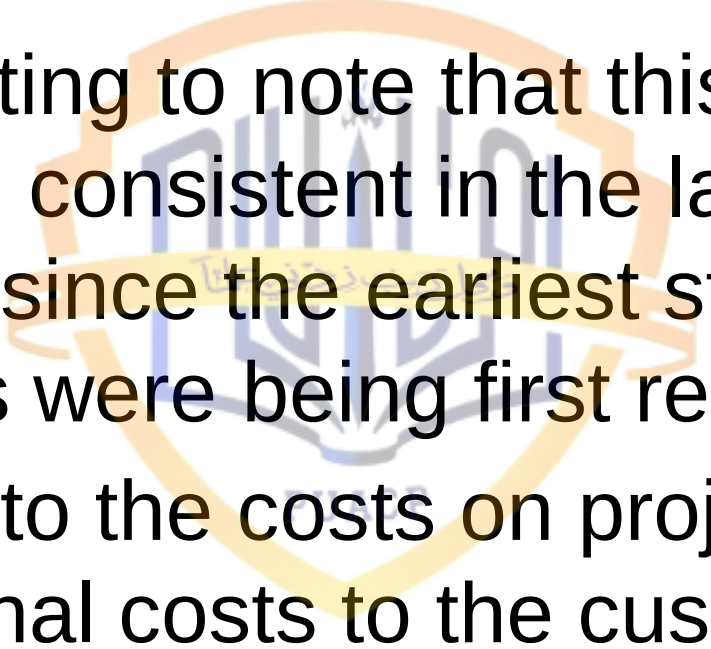
- 
- Several studies have confirmed the reduction in project costs when defects were removed earlier

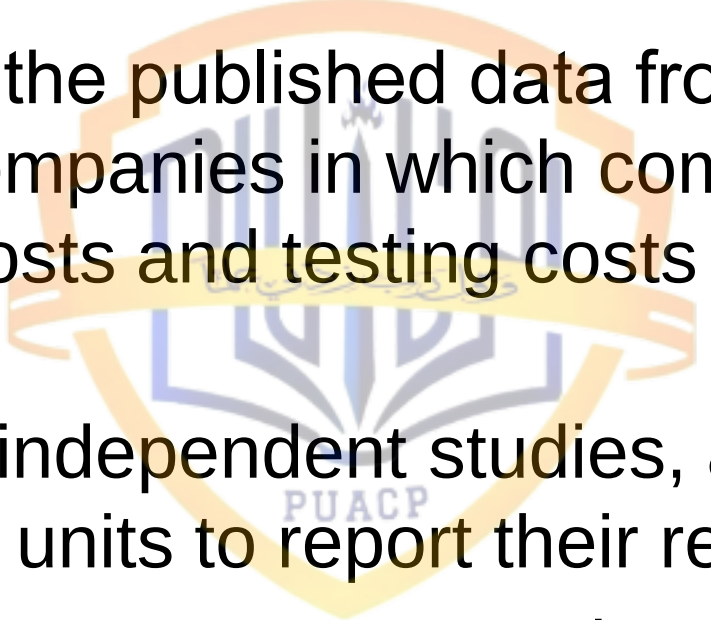


Defect Cost Relationship



Major Defect Removal Area

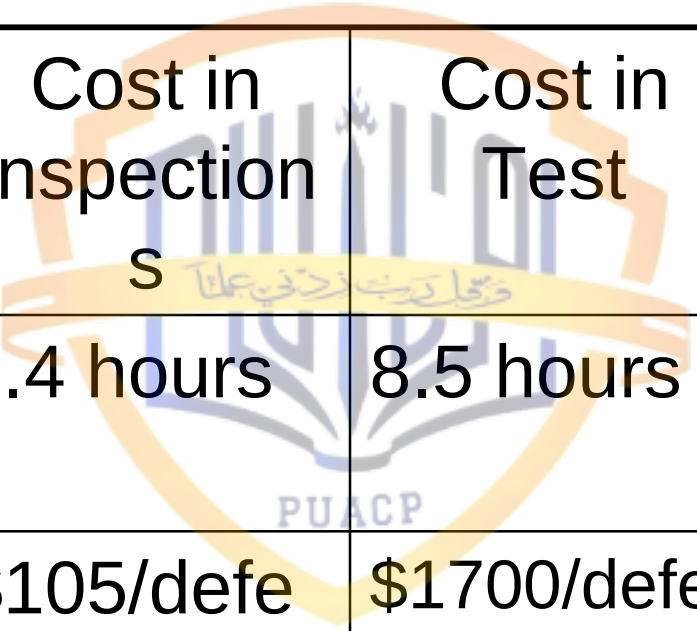
- 
- 
- It is interesting to note that this relationship has remain consistent in the last three decades – since the earliest studies when inspections were being first reported
 - In addition to the costs on project, there are additional costs to the customer for downtime, lost opportunity, etc., when defects are detected in maintenance

- 
- 
- Let's look at the published data from different studies of companies in which comparison of inspection costs and testing costs have been made
 - These were independent studies, and so they use different units to report their results
 - However, the pattern repeats that the cost of inspections is much lower than that of software testing

Reported Cost Relationship - 1

Company	Cost in Inspections	Cost in Test	Cost With Customer Discovery
IBM	\$48/defect	\$61-\$1030 / defect	\$1770 / defect
AT&T	1 unit	20 units	--
ICL	1.2-1.6 hours/defect	8.47 hours/defect	--

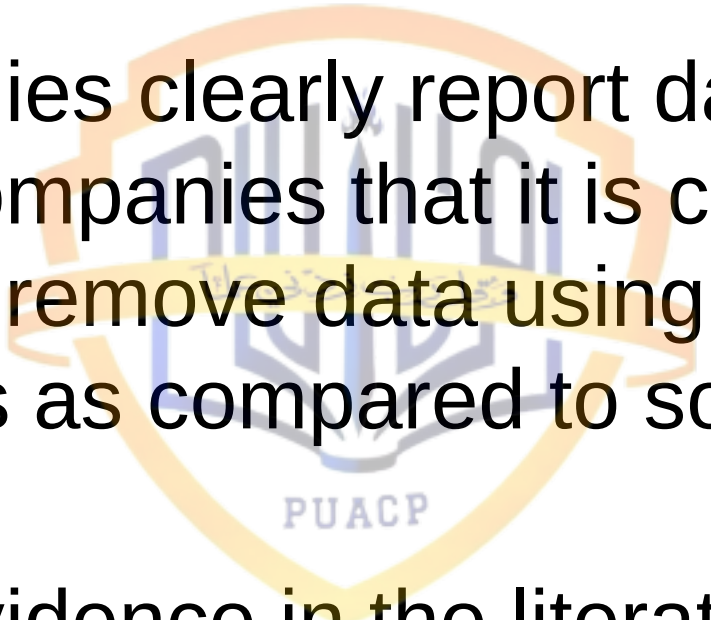
Reported Cost Relationship - 2

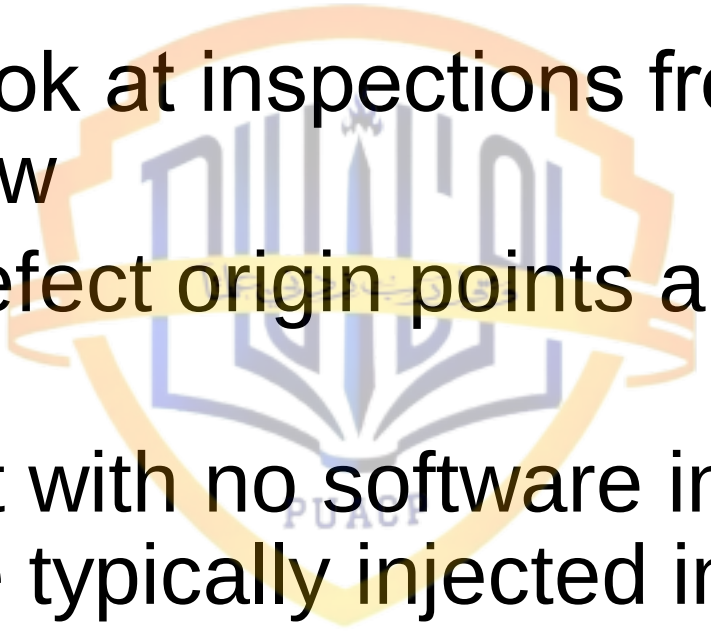


Company	Cost in Inspection	Cost in Test	Cost With Customer Discovery
AT&T	1.4 hours	8.5 hours	--
JPL	\$105/defect	\$1700/defect	--
IBM	1 unit	9 times more	117 times more

Reported Cost Relationship - 3

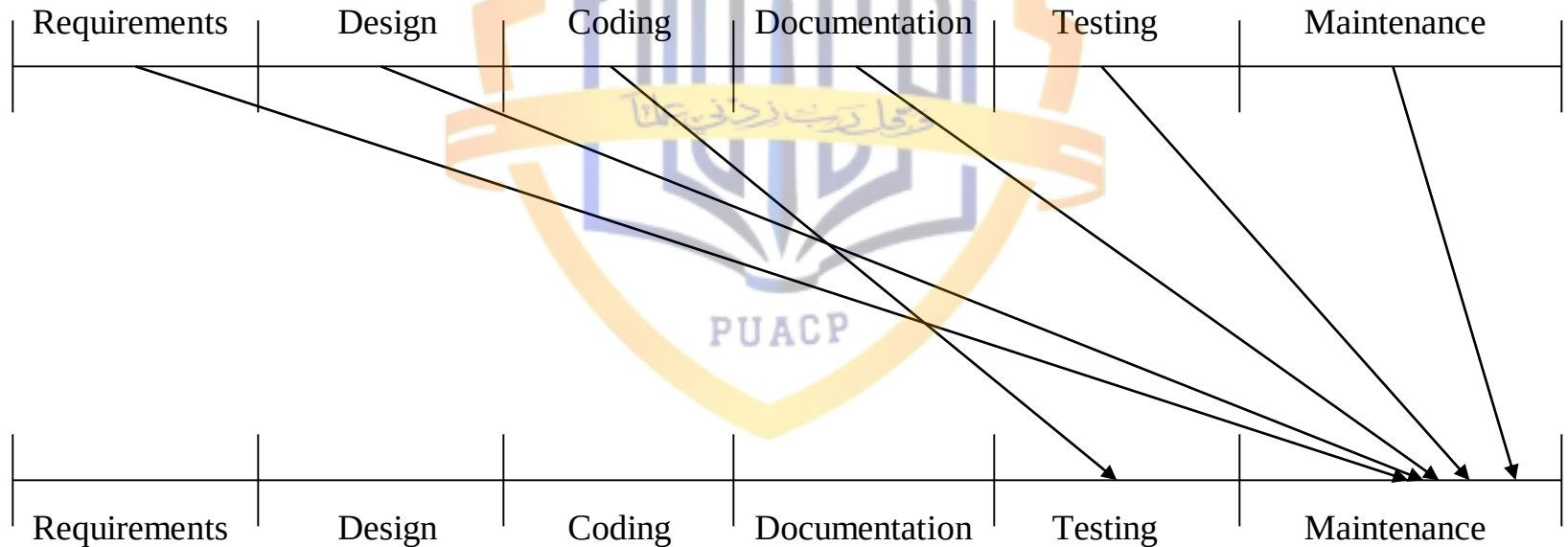
Company	Cost in Inspection S	Cost in Test	Cost With Customer Discovery
Shell	1 unit	30 units	--
Thorn EMI	1 unit	6.8-26 units	96 units
Applicon, Inc.	1 hour	--	30 hours
Infosys	1 unit	3 – 6 units	--

- 
- 
- These studies clearly report data from different companies that it is cheaper to detect and remove data using software inspections as compared to software testing
 - There is evidence in the literature that inspection offer significant return on investment even in their initial use

- 
- 
- Let' now look at inspections from another point of view
 - Relating defect origin points and defect discovery
 - In a project with no software inspections, defects are typically injected in the earlier activities and detected in later stages
 - As a result, we get a chaos zone

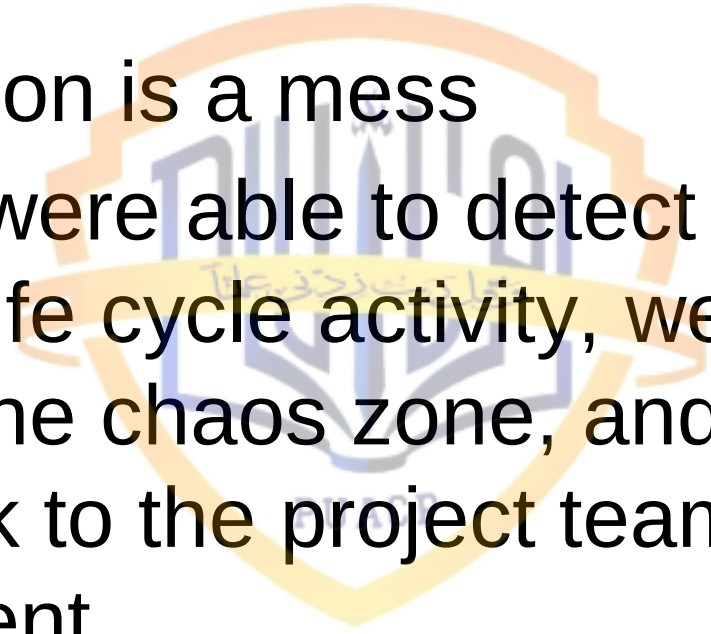
Defect Origins and Discovery Points Without Usage of Formal Inspections

Defect Origins



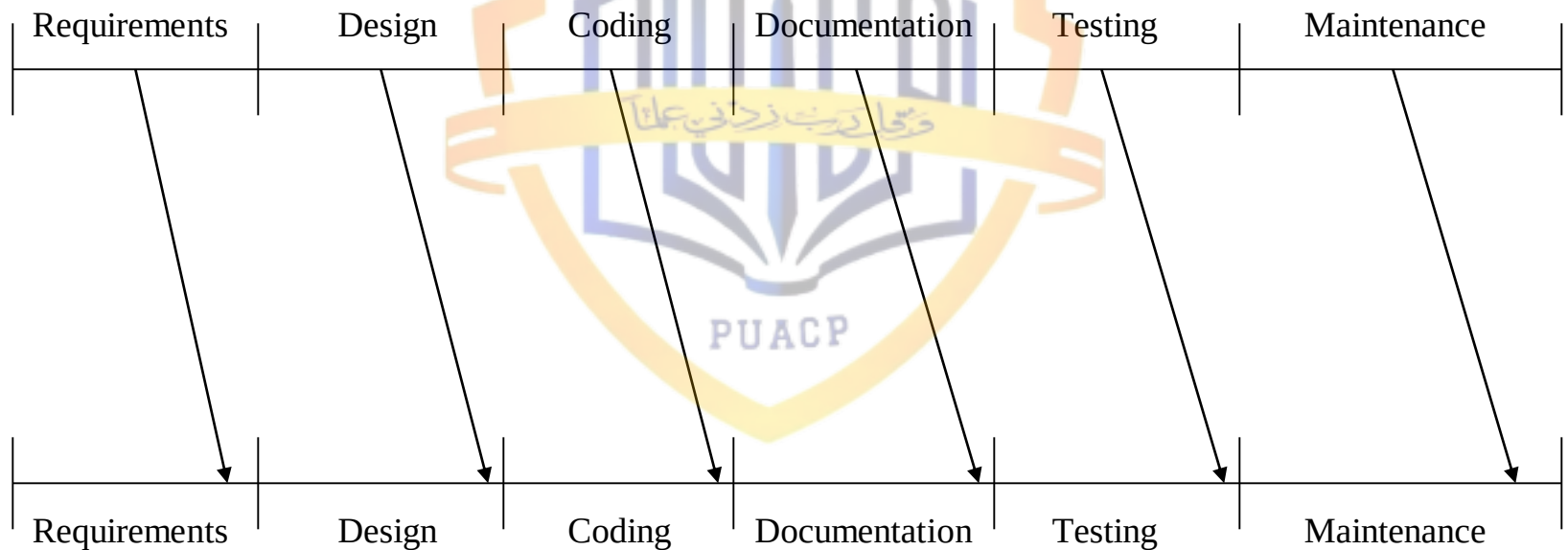
Defect Discovery

Chaos Zone

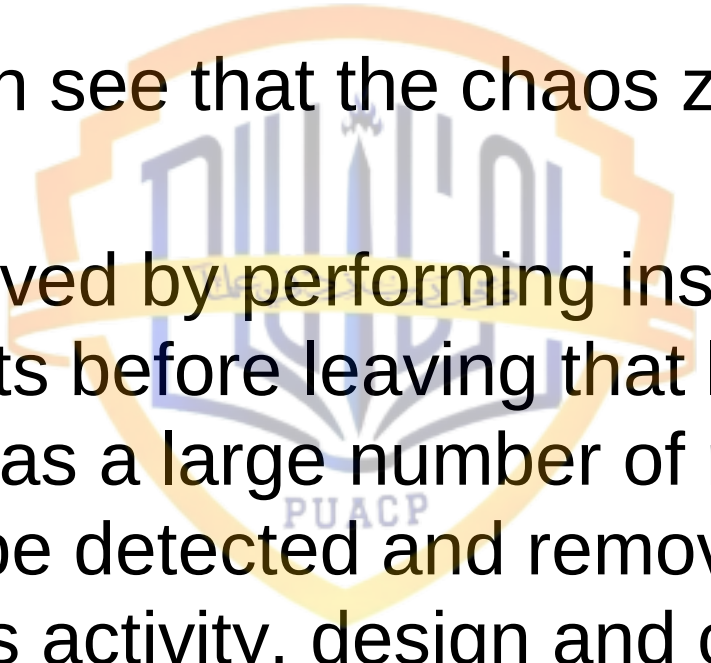
- 
- 
- This situation is a mess
 - If only we were able to detect defects in the same life cycle activity, we can eliminate the chaos zone, and bring some sanity back to the project team and project management
 - If we introduce software inspections, we can do that

Defect Origins and Discovery Points With Usage of Formal Inspections

Defect Origins



Defect Discovery

- 
- 
- Here you can see that the chaos zone has been eliminated
 - This is achieved by performing inspections on work products before leaving that life cycle activity, and as a large number of requirements defects will be detected and removed during the requirements activity, design and coding defects will be detected and removed during those activities, and so on

Why Isn't Everyone Using Inspections?

- Now we are convinced that inspections have a clear value independent of any model or standard for software development, so why isn't everyone using it?

Reasons for Not Using Inspections

- 1

- There is resistance to Inspections because people view them as if they are not easy to do well
- Management often views Inspections as an added cost, when in fact Inspections will reduce cost during a project
- Development of new tools and environments

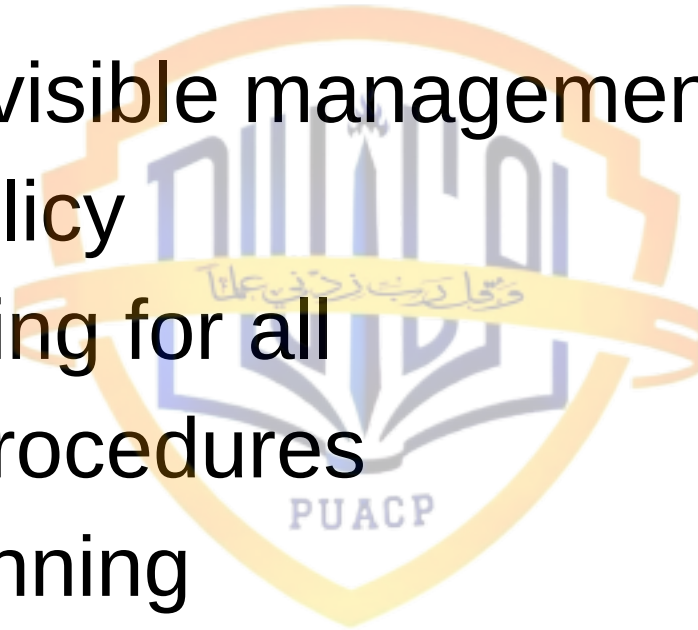
Reasons for Not Using Inspections

- 2

- Inspections are not the most enjoyable engineering task compared to designing and coding
- Inspections are labor intensive and low-tech
- Programmers/designers are possessive about the artifacts they create

Inspection Preconditions

- Clear and visible management support
- Defined policy
- Good training for all
- Effective procedures
- Proper planning
- Adequate resources



References

- Software Inspections by Ronald A. Radice, Tata-McGH, 2003 (Chapter 1)
- Software Quality: Analysis and Guidelines for Success by Capers Jones